

Recherche Textuelle 2

Algorithme de Morris-Pratt

Vocabulaire :

- Recherche par fenêtre glissante / Préfixe / Automate / l'algorithme de Morris-Pratt (MP) / Pseudo-code.

Prérequis :

- Connaissances générales sur le langage Python (notion de listes entre autres).
- Notion de graphe.
- Complexité d'un algorithme.



Objectifs :

- Implémenter un algorithme relativement performant pour la recherche de motif dans un texte : l'algorithme de Morris-Pratt.
- Implémenter un code en Python à partir d'un pseudo-code.
- Découvrir la notion de décorateur.

- Un moteur de recherche, tel que Google, référence plus de 130 000 milliards de pages web. Tu comprends l'intérêt d'utiliser un algorithme de recherche de motif efficace.
- Nous allons poursuivre l'étude des algorithmes de recherche de motif dans un texte. Nous allons chercher à implémenter un algorithme dont la complexité temporelle sera meilleure que celle de l'algorithme naïf vu à la séance précédente. On peut améliorer l'algorithme considérablement, en particulier en cherchant à décaler le motif de plus d'1 case à chaque fois.

I- OÙ FAIRE DES ÉCONOMIES ?

- Appelons **T** le texte dans lequel nous cherchons le motif **M** de longueur **m**.
- Nous effectuons une recherche par **fenêtre glissante** en faisant varier la position du motif par rapport au texte. Le début du motif sera placé à l'indice **i** du texte.

Exemple : on recherche le motif **abaa** en position $i=3$ dans le texte **aacabacabaabaa**

			i					i+m						
indice	0	1	2	3	4	5	6	7	8	9	10	11	12	13
T	a	a	c	a	b	a	c	a	b	a	a	b	a	a
M				a	b	a	a							
			0					m						

- Lorsqu'un **préfixe** du motif M (c'est-à-dire un ou plusieurs caractères qui sont en début de M), est présent à une position i de T :
 - On examine les caractères $T[i+1]$, $T[i+2]$, ... jusqu'à un éventuel échec mais on ne conserve pas cette information.
 - On examine donc de nouveau ces caractères pour tester la présence de M aux positions $i+1$, $i+2$...
- Reprenons l'exemple précédent : On vient d'étudier la possibilité de trouver le motif en position $i=3$ du texte. Le préfixe est en vert. La case rouge indique un échec.

indice	0	1	2	3	4	5	6	7	8	9	10	11	12	13
T	a	a	c	a	a	c	c	a	b	a	a	b	a	a
M				a	b	a	a							

Avant qu'il ne passe en position $i=4$, l'algorithme naïf a déjà examiné $T[4]$, $T[5]$ et $T[6]$.

$T[4] = b$ donc inutile de chercher abaa pour $i=4$.

$T[5] = a$ donc on pourrait chercher baa à partir de $i=6$ mais $T[6] \neq b$ donc il est inutile de continuer la recherche à cet endroit.

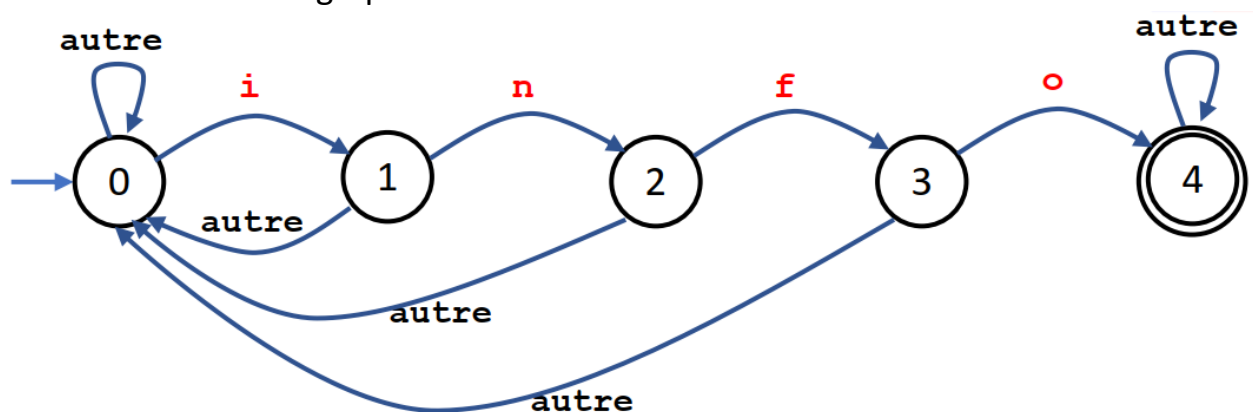
- On peut de la sorte éviter plusieurs tests inutiles (dits tests redondants).
On fait donc le choix de ne **jamais** lire plusieurs fois un même caractère de T et en cas d'échec dans le parcours du motif, on revient en arrière dans M autant que nécessaire.
- Pour cela, il ne suffit pas de connaître M sous forme d'une liste de caractères mais il faut analyser finement la structure de ce motif. Et puisque cela ne dépend pas de T, on peut le faire **une fois pour toutes** en amont de l'algorithme : c'est la phase de **pré-traitement**. Cette phase doit être si possible peu coûteuse, et en tous cas, accélérer la suite. Pour analyser le motif, on va se servir d'un **automate**.

II- UTILISATION D'UN AUTOMATE

A- Exemples introductifs

Prenons comme premier exemple le motif $M = \text{'info'}$.

- Voici l'idée : nous allons garder en mémoire où nous en sommes du mot **info**. Initialement, nous n'avons lu aucune des lettres de ce mot. Si à un moment, nous lisons **i**, nous garderons cette information en mémoire. Ensuite, si nous lisons juste après un **n**, nous garderons en mémoire que nous avons trouvé **in**, par contre, si nous lisons autre chose, nous revenons à l'état initial. En effet, un **i** suivi d'autre chose qu'un **n** ne peut pas servir à obtenir le mot voulu. On parcourt de la sorte tout le texte entré.
- Remarque : Le point important qui autorise cette méthode simple sur cet exemple est que toutes les lettres du mot cherché sont distinctes. Ainsi, si j'ai lu **inf** en partant de la k -ième lettre du texte, il est inutile de recommencer une recherche à partir de la $(k+1)$ -ième : nous savons que c'est un **n**, elle ne peut donc pas être le début de **info**. Nous pouvons poursuivre la recherche en position $(k+3)$.
- Maintenant, une méthode pratique pour retenir où nous en sommes de la lecture du motif est d'utiliser le graphe orienté suivant :



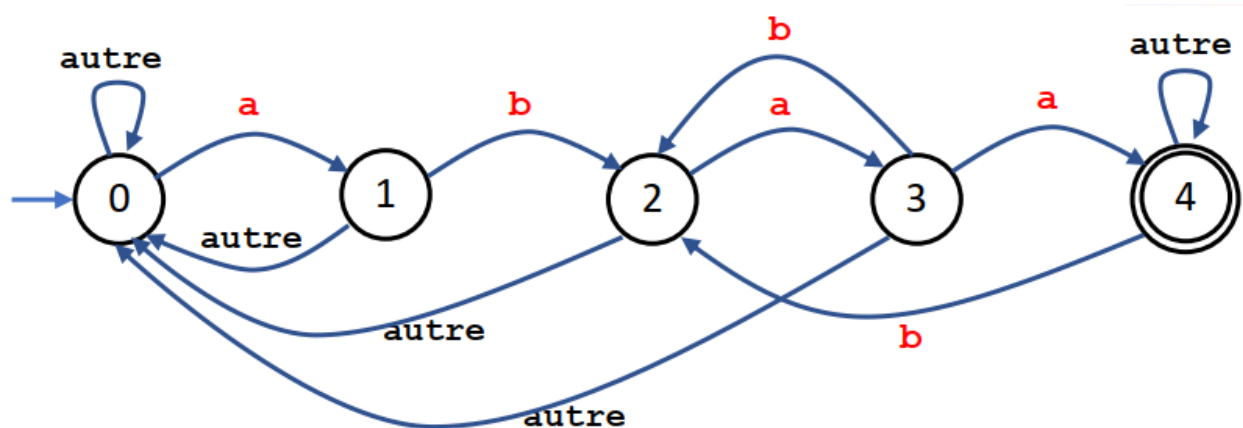
- On utilise ce graphe ainsi : on part du sommet 0, et à chaque lettre lue, on suit l'arête correspondante.
- Si à un moment nous arrivons dans le sommet 4, c'est que le mot cherché a été trouvé.
- Remarque : Pour être précis, il faut rajouter des transitions de 4 vers 4 : pour y rester jusqu'à la fin de la lecture.

Définition : Un tel graphe s'appelle un **automate** et ses sommets des états. La fonction qui à un état et un caractère associe l'état suivant s'appelle la **fonction de transition**.

- Il est assez facile de programmer l'automate de l'exemple précédent et on obtient bien un algorithme permettant de détecter si une chaîne de caractère contient le mot info en lisant une seule fois chaque caractère du texte. La recherche d'un motif dans un texte de taille n se fait donc en $O(n)$ mais le calcul de l'automate est plus coûteux (mais il n'est exécuté qu'une seule fois) et dépend surtout des lettres qui constituent le motif.

Maintenant, voyons comment adapter la méthode pour détecter le mot **M** = 'abaa'.

- Il y a une subtilité, due à la présence de la lettre **a** qui est redondante. Après le deuxième **a**, nous espérons encore un **a**, cependant si nous trouvons un **b**, nous ne devons pas revenir au départ mais au deuxième état.
- L'automate obtenu est :



- Il suffit désormais de suivre les transitions de l'automate pour le texte **T**, l'état final marque la présence du motif.

B- Calcul de l'automate

L'idée est donc, pour chaque caractère du motif **M** parcouru de gauche à droite, de déterminer la longueur du **plus grand préfixe** de **M** qui soit aussi **suffixe** de la sous-partie de **M** qui se termine par ce caractère.

- Par exemple, si on reprend notre motif **M** = 'abaa':
 - On associe 0 au premier caractère.
 - Le second caractère, qui est un **b**, n'est pas égal à **a**, préfixe de **M**, donc à ce caractère **b** sera aussi associé 0.
 - Le troisième caractère est un **a**, qui est aussi préfixe de **M**, et on ne peut pas trouver de préfixe de **M** plus long qui soit aussi suffixe du sous-motif se terminant par ce **a** (le préfixe **ab** est différent du suffixe **ba**). On associe donc 1 au troisième caractère de **M**.

➤ Enfin, le quatrième caractère qui est aussi un **a** sera associé à 1 pour les mêmes raisons que précédemment. On ne peut considérer pas '**abaa**' comme suffixe car il s'agit de l'ensemble de la chaîne.

- On peut donc présenter ces résultats sous la forme d'un tableau :

a	b	a	a
0	0	1	1

- Voici un autre exemple de motif **M** = '**1231234**' :

Sous-chaîne avec le plus grand préfixe en rouge	suffixe	Longueur de plus grand préfixe
1	/	0
12	/	0
123	/	0
1 23 1	1	1
1 23 1 2	12	2
1 23 1 23	123	3
1231234	/	0

On en déduit le tableau :

1	2	3	1	2	3	4
0	0	0	1	2	3	0

Exercice 1 : Soit le motif **M** = '**aaababaaa**'. Complète le tableau suivant en utilisant la règle précédente.

a	a	a	b	a	b	a	a	a

Exercice 2 : Soit le motif **M** = '**1010011**'. Complète le tableau suivant en utilisant la règle précédente.

1	0	1	0	0	1	1

II- ALGORITHME DE MORRIS-PRATT

A- Pré-traitement du motif

- Dans **l'algorithme de Morris-Pratt** (dit aussi algorithme **MP**), c'est la démarche effectuée dans la partie précédente qui va nous permettre de pré-traiter le motif.
- On cherche donc, pour toute position **j** dans **M**, la longueur **d** du plus long préfixe de **M** de la forme **M[j-d à j]**. Autrement dit la sous-chaîne préfixe qui soit identique à la sous-chaîne suffixe se terminant par l'élément en position **j**. On appellera la longueur de cette sous-chaîne **D[j]**.

Exemple :

j	0	1	2	3	4
M	a	a	b	a	a
D[j]	0	1	0	1	2

- Par conséquent :
 - Si **M[d] = M[j]** alors on prolonge le préfixe (éventuellement vide) déjà reconnu.
Exemple : Pour j=3, on avait d=1 car M[0]=M[3] (le plus grand préfixe est 'a'). Si on augmente d et j de 1 alors on a M[1]=M[4] : le préfixe est allongé. C'est 'aa'.
 - Sinon, on essaye de se rabattre sur un préfixe plus court, le meilleur est donné par **D[j]**.
 - S'il n'y a plus de préfixe plus court utilisable (**D[d]=0**), on repart du début de **M**.
- Algorithme de construction de la table de pré-traitement :
Entrée : un motif M de longueur m
Sortie : La table D utilisée dans l'algorithme KMP sous forme d'une liste.

Algorithme de pré-traitement du motif

```

Affecter à D[0] la valeur 0
d = 0 est la longueur du plus long préfixe déjà identifié
Pour chaque position j dans M sauf la première
    Tant que la longueur du préfixe déjà identifié ≠ 0 et que le caractère
    | courant du motif est ≠ du successeur du dernier caractère du préfixe en
    | cours d'identification (ce successeur est donné par M[d])
    |     Se rabattre sur un préfixe plus court
    FinTantque
    Si le préfixe peut être prolongé alors
    |     Incrémenter d de 1.
    FinSi
    Affecter à D[j] la valeur d
FinPour
Renvoyer D
  
```

Algorithme difficile et hors-programme
Il n'est pas à connaître par cœur.
Mais l'exercice 3 est un exercice de
passage d'un pseudo-code à une
implémentation en Python intéressant.

Exercice 3 : Complète la fonction Python, nommée `preTraitement(motif)` qui prend en paramètre un motif et qui renvoie la table D utilisée dans l'algorithme MP dans le fichier python élève de la séance du jour.

[Copie ta réponse ici et sauvegarde ensuite ton fichier pdf.]

Exercice 4 : Afin d'affiner ta compréhension de la fonction `preTraitement(motif)`, utilise l'application en ligne suivante : <http://www.pythontutor.com/> . Cela nécessite de prendre un motif comme exemple.

B- La recherche du motif dans l'algorithme Morris-Pratt

- Pendant la phase de recherche du motif dans le texte, en cas d'échec, on recule dans M indépendamment du prochain symbole du texte.
 - `D[j]` donne la position la plus à droite dans M d'où reprendre.
 - Si `D[j]=0`, c'est que rien ne convient et qu'il faut avancer dans T.



James H. Morris



Vaughan Pratt

Exercice 5 : Complète l'algorithme de MP suivant, écrit en pseudo-code, afin qu'il renvoie la première occurrence du motif M dans le texte T :

Algorithme de recherche Morris-Pratt

```

i ← 0
j ← 0
m ← longueur(M)
n ← longueur(T)
Tant que i < n
|
|   Si T[i] = M[j] alors
|   |   i ← .....
|   |   j ← .....
|   Sinon
|   |   Si j > 0 alors
|   |   |   j ← ...].....
|   |   Sinon
|   |   |   i ← i + 1
|   |   FinSi
|   FinSi
|   Si ..... alors
|   |   Renvoyer «le motif a été trouvé en position :», .....
|   FinSi
FinTantque
Renvoyer « motif absent »

```

Exercice 6 :

- 1) Complète la fonction Python, nommée `rechercheMP(texte, motif)` qui prend en paramètre un motif et le texte et qui renvoie l'indice de la première occurrence du motif trouvé dans le texte.
- 2) Modifie ta fonction pour que la fonction renvoie la **liste** des indices de toutes les occurrences du motif trouvé.

[Copie ta réponse ici et sauvegarde ensuite ton fichier pdf.]

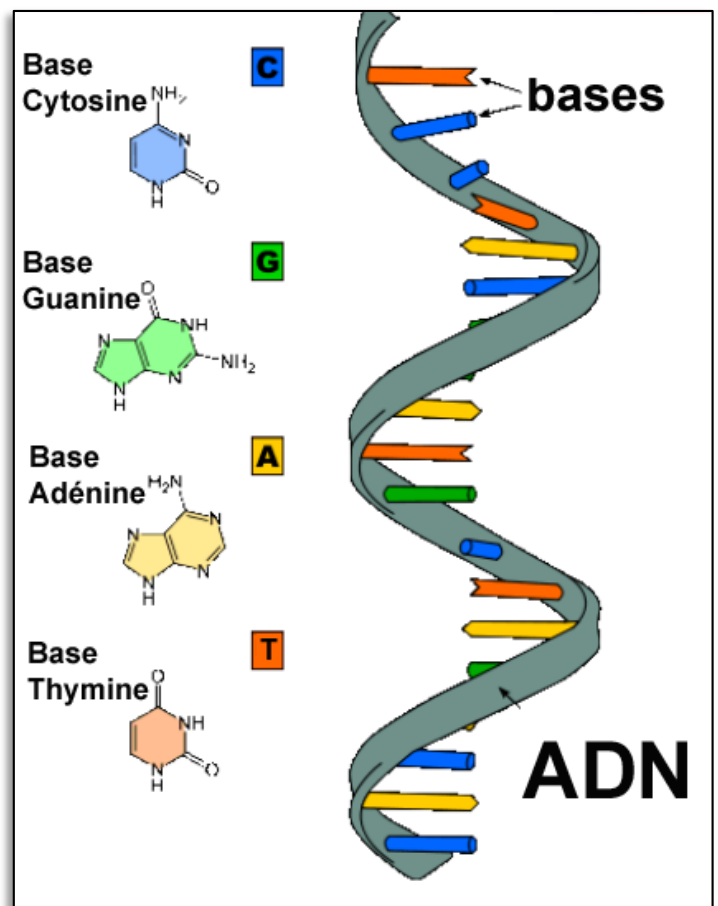
Exercice 7 : Application à la recherche biologique

Le virus responsable de la maladie Covid-19 s'appelle le SARS-CoV-2.

Son génome a été rapidement séquencé par des chercheurs chinois, il s'agit d'une molécule d'ARN d'environ 30 000 bases (on rappelle que les bases sont notées A, T, G et C) contenant 15 gènes.

À titre de comparaison, notre génome est sous forme d'une double hélice d'ADN d'une taille d'environ 3 milliards de bases et il contient près de 30 000 gènes.

Le site américain NCBI ([cliquer ici](#)) est un site hébergeant une banque de génomes, de gènes et de protéines. Cette banque est régulièrement complétée par des laboratoires du monde entier et par des centres de séquençage.



Le dossier intitulé « séquençage » contient, entre autres, le séquençage de 4 gènes (les gènes ASP, HIV2gp3, ORF6 et S) et celui du génome du SARS-CoV-2. En utilisant l'algorithme de Morris-Pratt, identifie parmi les 4 gènes ceux qui sont présents dans le génome du SARS-CoV-2.

[Copie ta réponse ici et sauvegarde ensuite ton fichier pdf.]

Exercice 8 : Comparaison du temps d'exécution des deux algorithmes (naïf et MP).

Le décorateur Timer calcule et affiche la durée d'exécution d'une fonction.

L'algorithme naïf est particulièrement mauvais si le texte répète souvent une grosse partie du motif.

Teste le temps d'exécution des deux algorithmes sur le fichier Zéro.txt (composé d'un million de 0) et le motif 0000000000000000000001.

[Copie ta réponse ici et sauvegarde ensuite ton fichier pdf.]